

INTERNET DAS COISAS

INSTALAÇÃO, CONFIGURAÇÃO E MODIFICAÇÃO DA ESP-32

UNIVERSIDADE FEDERAL DO CEARÁ

AURELICE, ANDERSON, PEDRO
WILLY, TALYSON, YNGRID

Agenda

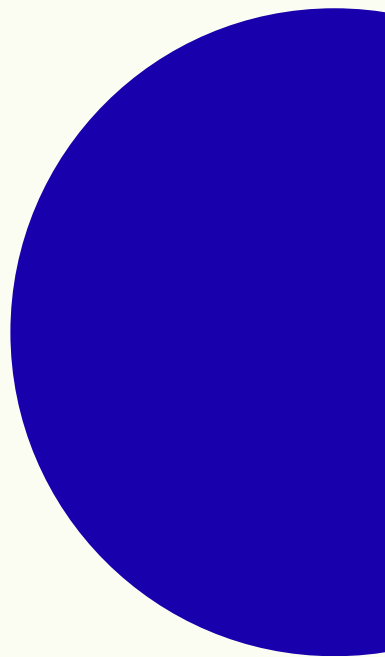
Objetivo

Ferramentas utilizadas

Instalação e Configuração

Demonstração prática

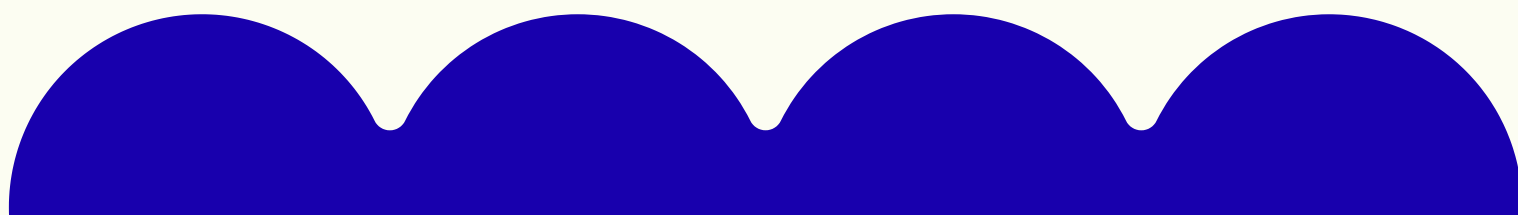
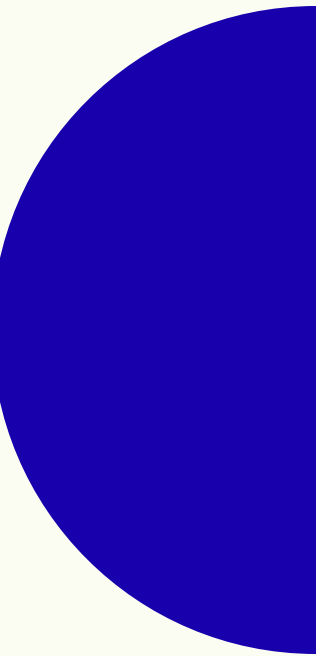
Conclusão



Objetivo

Objetivo

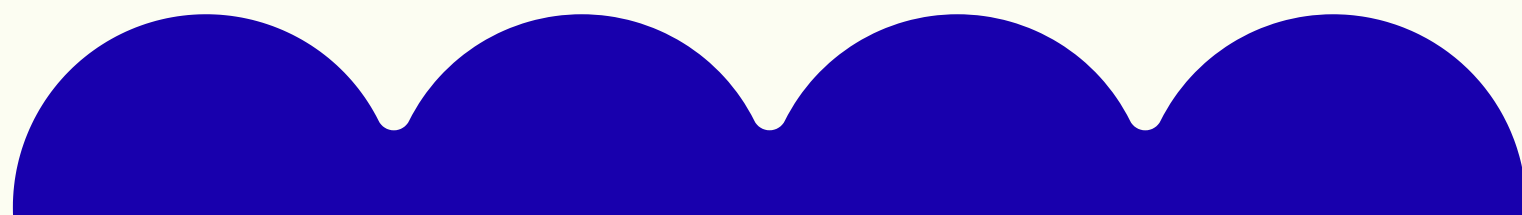
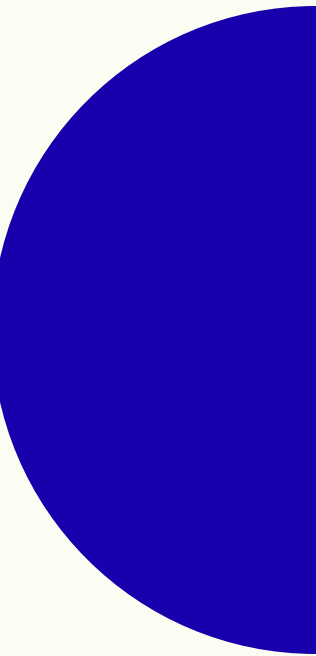
Configurar o ambiente de desenvolvimento para a ESP32-CAM utilizando o ESP-IDF e realizar testes básicos de programação e comunicação com a placa, incluindo execução de um programa inicial, controle de GPIO e utilização do Wi-Fi.



Ferramentas utilizadas

Ferramentas utilizadas

- ESP32-CAM – placa principal com microcontrolador ESP32
- Cabo USB – conexão entre a placa e o computador
- ESP-IDF v6.1 – framework oficial utilizado para desenvolver e compilar os programas
- ESP-IDF PowerShell – terminal utilizado para executar os comandos
- Windows 11 – sistema operacional utilizado durante os testes



Instalação e Configuração

Instalação

Primeiro, foi preciso acessar a página oficial da ESPRESSIF e localizar o botão de download, como na imagem abaixo:

Step 2: Install the EIM

You can install the EIM using one of the following methods:

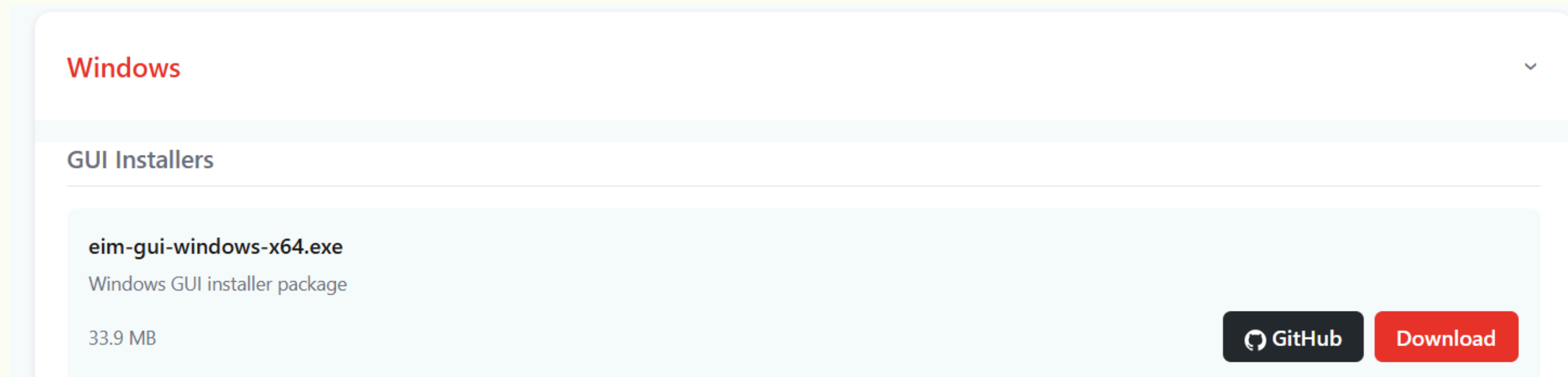
- Download the EIM



You can choose either an online or offline installer, available in Graphical User Interface (GUI) or Command Line Interface (CLI) versions.

Instalação

Ao clicar no botão, o usuário é levado até outra tela, onde é possível localizar o download do arquivo necessário.



Instalação

Selecionamos a opção “easy installation” para instalação de drivers, em seguida, selecionamos a dashboard.

Welcome to **ESP-IDF** Installation Manager

Manage your existing installations or install a new version:



Manage Installations

View and manage your 1 installed ESP-IDF version(s)

Open Dashboard



New Installation

Install ESP-IDF development environment

Start Installation

Instalação

Ao abrirmos a dashboard, é apresentado as versões instaladas, podemos clicar no primeiro ícone para abrirmos o terminal e iniciar a configuração

ESP-IDF Version Management

Installed Versions

v6.1

📁 C:\esp\v6.1\esp-idf



Configuração

Primeiro criamos uma pasta para o projeto e copiamos de outro diretório, escolhemos o “hello-world”, em seguida entramos na pasta pelo terminal com “cd hello-world”

```
IDF_v6.1_Powershell
(venv) PS C:\Users\ander\OneDrive\Desktop> cd $env:USERPROFILE
(venv) PS C:\Users\ander> mkdir esp

Diretório: C:\Users\ander

Mode                LastWriteTime         Length Name
----                -
d-----           21/09/2026   16:53         esp

(venv) PS C:\Users\ander> cd esp
(venv) PS C:\Users\ander\esp> Copy-Item -Recurse "$env:IDF_PATH\examples\get-started\hello_world" .
(venv) PS C:\Users\ander\esp> cd hello_world
(venv) PS C:\Users\ander\esp\hello_world> dir

Diretório: C:\Users\ander\esp\hello_world

Mode                LastWriteTime         Length Name
----                -
d-----           21/09/2026   16:53         main
-a-----           01/09/2026   11:08         370 CMakeLists.txt
-a-----           01/09/2026   11:08      1888 pytest_hello_world.py
-a-----           01/09/2026   11:08      2537 README.md
-a-----           01/09/2026   11:08          0 sdkconfig.ci
```

Configuração

Em seguida, precisamos conectar a placa a máquina e fazer no terminal com que torne a placa o alvo de busca

```
IDF_v6.1_Powershell
(venv) PS C:\Users\ander\esp\hello_world> idf.py set-target esp32
Adding "set-target"'s dependency "fullclean" to list of commands with default set of options.
Executing action: fullclean
Build directory 'C:\Users\ander\esp\hello_world\build' not found. Nothing to clean.
Executing action: set-target
Set Target to: esp32, new sdkconfig will be created.
Running cmake in directory C:\Users\ander\esp\hello_world\build
Executing "cmake -G Ninja -B C:\Users\ander\esp\hello_world\build -DPYTHON_DEPS_CHECKED=1 -DPYTHON=C:\Espressif\tools\python\v6.1\venv\Scripts\python.exe -DESP_PLATFORM=1 -DIDF_TARGET=esp32 -DCCACHE_ENABLE=True C:\Users\ander\esp\hello_world"...
-- Found Git: C:/Program Files/Git/cmd/git.exe (found version "2.55.0.windows.5")
-- Minimal build - ON
-- ccache will be used for faster recompilation
-- The C compiler identification is GNU 15.2.0
-- The CXX compiler identification is GNU 15.2.0
-- The ASM compiler identification is GNU
-- Found assembler: C:/Espressif/tools/xtensa-esp-elf/esp-15.2.0_20251204/xtensa-esp-elf/bin/xtensa-esp32-elf-gcc.exe
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/Espressif/tools/xtensa-esp-elf/esp-15.2.0_20251204/xtensa-esp-elf/bin/xtensa-esp32-elf-gcc.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/Espressif/tools/xtensa-esp-elf/esp-15.2.0_20251204/xtensa-esp-elf/bin/xtensa-esp32-elf-g++.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- git rev-parse returned 'fatal: not a git repository (or any of the parent directories): .git'
```


Configuração

Aqui começamos a compilação da esp

```
IDF_v6.1_Powershell
(venv) PS C:\Users\ander\esp\hello_world> idf.py build
Executing action: all (aliases: build)
Running ninja in directory C:\Users\ander\esp\hello_world\build
Executing "ninja all"...
[4/508] Generating ../../partition_table/partition-table.bin
Partition table binary generated. Contents:
*****
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
nvs,data,nvs,0x9000,24K,
phy_init,data,phy,0xf000,4K,
factory,app,factory,0x10000,1M,
*****
[503/508] Generating binary image from built executable
esptool v5.3.1
Creating ESP32 image...
Merged 3 ELF sections.
Successfully created ESP32 image.
Generated C:/Users/ander/esp/hello_world/build/hello_world.bin
[504/508] C:\WINDOWS\system32\cmd.exe /C "cd /D C:\Users\ander\esp\hello_wor...le/partition-table.bin C:/Users/ander/esp/hello_world/build/hello_world.bin"
hello_world.bin binary size 0x1ebe0 bytes. Smallest app partition is 0x100000 bytes. 0xe1420 bytes (88%) free.
[505/508] Performing configure step for 'bootloader'
-- Found Git: C:/Program Files/Git/cmd/git.exe (found version "2.55.0.windows.5")
-- Minimal build - OFF
-- The C compiler identification is GNU 15.2.0
-- The CXX compiler identification is GNU 15.2.0
-- The ASM compiler identification is GNU
-- Found assembler: C:/Espressif/tools/xtensa-esp-elf/esp-15.2.0_20251204/xtensa-esp-elf/bin/xtensa-esp32-elf-gcc.exe
-- Detecting C compiler ABI info
```


Configuração

Em seguida é necessário descobrir a porta que a placa está conectada, para isso se acessa o gerenciador de dispositivos (Win+X) e procura por portas, nesse caso como da imagem, nossa porta é COM3

```
IDF_v6.1_Powershell
(venv) PS C:\Users\ander\esp\hello_world> idf.py -p COM3 flash
Adding "flash"'s dependency "all" to list of commands with default set of options.
Executing action: all (aliases: build)
Running ninja in directory C:\Users\ander\esp\hello_world\build
Executing "ninja all"...
[1/4] C:\WINDOWS\system32\cmd.exe /C "cd /D C:\Users\ander\esp\hello_world\build && C:\Users\ander\esp\hello_world\build\hello_world.bin"
hello_world.bin binary size 0x1ebe0 bytes. Smallest app partition is 0x100000 bytes. 0xe1420 bytes (88%) free.
[1/1] C:\WINDOWS\system32\cmd.exe /C "cd /D C:\Users\ander\esp\hello_world\build && C:\Users\ander\esp\hello_world\build\bootloader\bootloader.bin"
Bootloader binary size 0x6640 bytes. 0x9c0 bytes (9%) free.
[4/4] Completed 'bootloader'Executing action: flash
Running ninja in directory C:\Users\ander\esp\hello_world\build
Executing "ninja flash"...
[1/5] C:\WINDOWS\system32\cmd.exe /C "cd /D C:\Users\ander\esp\hello_world\build && C:\Espressif\tools\python\v6.1\venv\Scripts\python.exe C:/esp/v6.1/esp-idf/components/partition_table/check_sizes.py --offset 0x8000 partition --type app C:/Users/ander/esp/hello_world/build/partition_table/partition-table.bin C:/Users/ander/esp/hello_world/build/hello_world.bin"
hello_world.bin binary size 0x1ebe0 bytes. Smallest app partition is 0x100000 bytes. 0xe1420 bytes (88%) free.
[2/5] Performing build step for 'bootloader'
[1/1] C:\WINDOWS\system32\cmd.exe /C "cd /D C:\Users\ander\esp\hello_world\build\bootloader && C:\Espressif\tools\python\v6.1\venv\Scripts\python.exe C:/esp/v6.1/esp-idf/components/partition_table/check_sizes.py --offset 0x8000 bootloader 0x1000 C:/Users/ander/esp/hello_world/build/bootloader/bootloader.bin"
Bootloader binary size 0x6640 bytes. 0x9c0 bytes (9%) free.
[3/5] No install step for 'bootloader'
[4/5] Completed 'bootloader'
[4/5] Running utility command for flash
esptool --chip esp32 -p COM3 -b 460800 --before=default-reset --after=hard-reset write-flash --flash-mode dio --flash-frequency 40m --flash-size 2MB 0x1000 bootloader/bootloader.bin 0x8000 partition_table/partition-table.bin 0x10000 hello_world.bin --skip-flashed
```

Configuração

Por fim, o programa é executado com sucesso

```
IDF_v6.1_Powershell
(venv) PS C:\Users\ander\esp\hello_world> idf.py -p COM3 monitor
Executing action: monitor
Running idf_monitor in directory C:\Users\ander\esp\hello_world
Executing "C:\Espressif\tools\python\v6.1\venv\Scripts\python.exe C:\esp\v6.1\esp-idf\tools\idf_monitor.py -p COM3 -b 115200 --toolchain-prefix xtensa-esp32-elf- --target esp32 --revision 0 C:\Users\ander\esp\hello_world\build\hello_world.elf C:\Users\ander\esp\hello_world\build\bootloader\bootloader.elf --force-color -m 'C:\Espressif\tools\python\v6.1\venv\Scripts\python.exe' 'C:\esp\v6.1\esp-idf\tools\idf.py' '-p' 'COM3'"...
--- Warning: GDB cannot open serial ports accessed as COMx
--- Using \\.COM3 instead...
--- esp-idf-monitor 1.9.0 on \\.COM3 115200
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
ets Jul 29 2019 12:21:46

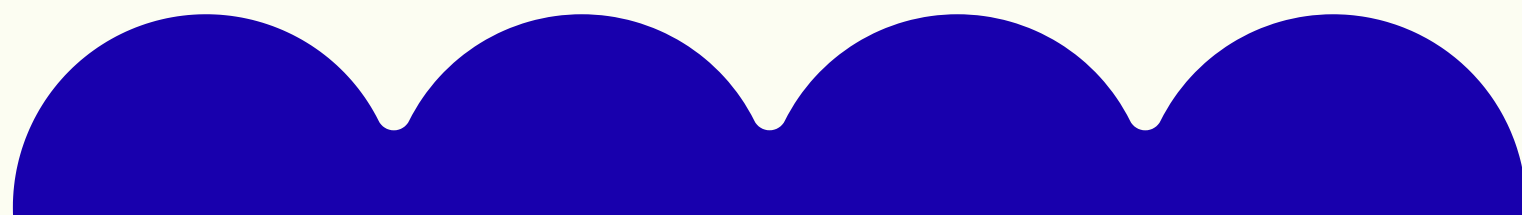
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0040,len:6272
load:0x40078000,len:15824
load:0x40080400,len:3988
--- 0x40080400: _invalid_pc_placeholder at C:/esp/v6.1/esp-idf/components/xtensa/xtensa_vectors.S:2259
entry 0x40080644
--- 0x40080644: call_start_cpu0 at C:/esp/v6.1/esp-idf/components/bootloader/subproject/main/bootloader_start.c:27
I (27) boot: ESP-IDF v6.1 2nd stage bootloader
I (27) boot: compile time Sep 21 2026 17:08:57
I (27) boot: Multicore bootloader
I (29) boot: chip revision: v3.1
I (31) boot.esp32: SPI Speed      : 40MHz
I (35) boot.esp32: SPI Mode      : DIO
I (39) boot.esp32: SPI Flash Size : 2MB
```


Demonstração prática com modificação

Conclusão

Conclusão

A utilização da ESP32-CAM com o ESP-IDF permitiu compreender o processo básico de desenvolvimento para sistemas embarcados, desde a configuração do ambiente até a compilação, gravação e execução de programas na placa. Os testes realizados com o Hello World, controle de GPIO e varredura de redes Wi-Fi demonstraram o funcionamento dos principais recursos iniciais da ESP32. A atividade também serviu como base para aplicações futuras envolvendo câmera, conexão Wi-Fi e desenvolvimento de sistemas IoT.



Agradecemos pela atenção

aurelice@alu.ufc.br

andersoncelestino@alu.ufc.br

talysonfreitas@alu.ufc.br

pedrowilly@alu.ufc.br

ynggridcoelho@alu.ufc.br